

ML Orale

Davide Cuna

August 2025

Contents

1	Regressione	3
1.1	Supervised Learning	3
1.2	Linear Regression	3
1.2.1	Formule	3
2	Working with Text Data	4
2.1	Preprocessing	4
2.2	Bag of Words	4
2.3	Term Frequency Inverse Document Frequency	4
2.4	One-hot Vector	4
2.5	Word Embeddings	4
3	Neural Network	4
3.1	Funzione di attivazione	5
3.2	Loss Function	5
3.3	Flusso dell'Apprendimento	5
4	SVM	5
4.1	Margini e Distanza tra i Margini	5
4.2	Cost Function delle SVM	6
4.3	Regularization Term	6
4.4	Kernel Gaussiani	6
4.5	Gaussian Radius	6
4.6	Scelta dei landmarks	6
5	Regularization	6
5.1	Holdout Method	6
5.1.1	K-fold Cross Validation	6
5.2	Overfitting e Underfitting	7
5.3	Ridge Regression	7
5.4	Regolarizzazione della Logistic Regression	7
5.5	Learning Curve	7
5.6	Fix da provare	7
6	Dimensionality Reduction	7
6.1	Differenza Linear Regression e PCA	7
6.2	Funzionamento Algoritmo	7
6.3	Scelta di k	8

1 Regressione

1.1 Supervised Learning

Approccio fondamentale al machine learning che consiste nel training di un modello usando dei dati "etichettati". Ogni input è appaiato con il corretto output; col tempo il modello si sistema per minimizzare gli errori ed aumentare la accuracy.

Si segue il seguente approccio:

- **Training Data:** viene fornito al modello un training dataset che includa input (features) ed output (labels)
- **Learning Process:** l'algoritmo processa i dati di training, imparando la relazione tra input ed output modificando i parametri in modo da minimizzare la differenza tra predictions e le label effettive

Una volta terminato il training il modello viene valutato tramite un test dataset per misurare accuracy e performance.

Vantaggi e Svantaggi :

- + eccelle nel predire patterns e fare scelte basate sui dati
- + buono per applicazioni come image recognition, natural language processing etc
- overfitting (quando il modello impara troppo dal training data inclusi dettagli poco importanti). Ragioni per overfitting:
 - alta varianza e bias basso
 - modello troppo complesso
 - training data troppo piccolo
- feature engineering: estrarre feature rilevanti è molto importante ma può richiedere molto tempo
- il bias del modello può risultare in predizioni non vere
- questo sistema dipende molto nell'avere dati già etichettati

1.2 Linear Regression

Tipo di algoritmo di supervised learning che impara da un dataset etichettato mappando i data points con linear functions ottimizzate utilizzabili per fare predizioni su nuovi dataset.

Perchè la linear regression è importante?

- Semplice ed interpretabile
- Capacità predittive
- Efficiente in termini di computazione

Best Fit Line Nella linear regression la best fit line è una linea retta (o iperpiano nel caso delle polinomiali) che rappresenta in maniera più accurata la relazione tra gli input e gli output minimizzando la differenza tra predizioni ed i data points effettivi.

Obiettivo Linear Regression l'obiettivo della linear regression consiste nella minimizzazione della funzione di costo, per farlo si usa la tecnica del gradien descent.

1.2.1 Formule

- Funzione di costo $J(\theta) = \frac{1}{2m} \cdot \sum_{i=1}^m (h_{\theta}(x_i) - y_i)^2$

2 Working with Text Data

2.1 Preprocessing

- Lowercasing
- Tokenization (divisione del testo in parole o sottoparole)
- Stopword Removal (eliminazione delle parole comuni poco informative)
- Stemming / Lemmatization (riduzione delle parole alla radice)
- Punctuation Removal

2.2 Bag of Words

Tecnica per la rappresentazione di un documento come un vettore di frequenze delle parole. Non considera l'ordine delle parole ma solo la presenza e la frequenza.

Limiti BoW

- Sparsità nei vettori (molti zeri)
- Nessuna informazione semantica o di ordine

2.3 Term Frequency Inverse Document Frequency

Tecnica che migliora il BoW pesando le parole in base alla loro importanza:

- Parole frequenti in pochi documenti → peso alto
- Parole comuni in tutti documenti → peso basso

2.4 One-hot Vector

- Ogni parola è rappresentata da un vettore di lunghezza pari al vocabolario
- Solo una posizione è '1' il resto è tutto '0'

Limitazioni

- Vettori molto sparsi
- Nessuna relazione semantica

2.5 Word Embeddings

- Rappresentazioni **dense e continue** in uno spazio vettoriale a bassa dimensione
- Parole semanticamente simili hanno vettori vicini
- Ottenuti tramite reti neurali o modelli predittivi

3 Neural Network

Modelli matematici composti da neuroni artificiali organizzati in più strati:

- **Strato di input:** riceve i dati grezzi
- **Strato nascosto:** trasformano l'informazione tramite pesi e funzione di attivazione
- **Strato di output:** produce il risultato finale (classificazione, regressione, ...)

Ogni connessione tra neuroni ha un peso che rappresenta l'importanza dell'input. Il neurone calcola una combinazione lineare degli input e la passa attraverso una funzione di attivazione per introdurre la non linearità. L'allenamento si basa su una tecnica di backpropagation che regola i pesi minimizzando una loss function con metodi di ottimizzazione come gradient descent.

3.1 Funzione di attivazione

- **Sigmoide:** Output compreso tra 0 e 1 quindi utile per le classificazioni binarie. **Problema:** saturazione del gradiente (per valori grandi o negativi tende a 0) rallentando il tasso di apprendimento

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- **ReLU:** Molto usata nelle reti profonde, semplice ed evita la saturazione positiva. **Problema:** dei neuroni si bloccano a 0 come output (dying ReLU)

$$f(x) = \max(0, x)$$

- **SoftMax:** Trasforma un vettore di valori reali in una distribuzione di probabilità quindi usata nello strato di output per la classificazione multiclasse.

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

3.2 Loss Function

Le Loss Function misurano l'errore tra predizione e valore atteso, guidando l'ottimizzazione dei pesi.

- **MSE - Mean Squared Error** Usata nella regressione. Penalizza fortemente errori molto grandi, poco adatta alla classificazione

$$MSE = \frac{1}{n} \cdot \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- **BCE - Binary Cross Entropy Loss** Usata per la classificazione binaria, funziona bene con una sigmoide in output

$$L = -\frac{1}{n} \cdot \sum_{i=1}^n [y_i \cdot \ln(\hat{y}_i) + (1 - y_i) \cdot \ln(1 - \hat{y}_i)]$$

- **CE - Cross Entropy Loss** Generalizza la BCE a più classi (con C il numero di classi)

$$L = -\sum_{i=1}^C y_i \cdot \ln(\hat{y}_i)$$

3.3 Flusso dell'Apprendimento

1. Input dei dati
2. ForwardPropagation (calcolo output e attivazioni)
3. Calcolo della loss
4. BackPropagation (calcolo del gradiente)
5. Aggiornamento dei pesi tramite un ottimizzatore (Gradient Descent)
6. Ripeti fino alla convergenza

4 SVM

Sono algoritmi di apprendimento supervisionato usati per classificazione (e ogni tanto regressione). L'idea è quella di trovare un iperpiano di separazione che divide i dati di due classi con il margine più ampio possibile. I Support Vector invece sono i punti del dataset che si trovano sul bordo del margine e determinano la posizione dell'iperpiano.

4.1 Margini e Distanza tra i Margini

In una SVM lineare si cerca un iperpiano $w \cdot x + b = 0$.

La distanza di un punto x dall'iperpiano è $d(x) = \frac{|w \cdot x + b|}{||w||}$

I margini sono le due linee parallele che passano per i support vector:

- $w \cdot x + b = +1$
- $w \cdot x + b = -1$

La distanza tra i margini (cioè lo "spazio vuoto" fra le due linee) è: $Distanza = \frac{2}{\sqrt{w^T w}}$

4.2 Cost Function delle SVM

Il problema di ottimizzazione si può formulare come

$$\min_{w,b} \frac{1}{2} \cdot \|w\|^2 + C \cdot \sum_{i=1}^n \epsilon_i$$

dove C è un regularization term che bilancia margine ampio e errori di classificazione, mentre ϵ sono le slack variables.

$$\min_{w,b} \frac{w^T w}{2} \quad \text{soggetta a: } y_i(w^T \cdot x_i + b) \geq 1 \quad \forall x_i$$

4.3 Regularization Term

- Se C è **grande** la SVM punisce fortemente gli errori, margine più piccolo ma la classificazione più accurata sui dati di training (rischiando l'overfitting)
- Se C è **piccolo** la SVM accetta più errori, ha un margine più ampio e c'è una migliore generalizzazione.

4.4 Kernel Gaussiani

Quando i dati non sono linearmente separabili, si usano kernel per proiettare i punti in uno spazio di dimensioni maggiori. Il kernel gaussiano misura la similarità tra due punti x e l :

$$K(x, l) = \exp\left(-\frac{\|x - l\|^2}{2\sigma^2}\right)$$

- Restituisce valori compresi tra 0 e 1
- Se x è vicino al landmark, allora la similarità è prossima ad 1
- Se x è lontano dal landmark, allora la similarità si avvicinerà a 0

4.5 Gaussian Radius

- σ controlla l'ampiezza della campana gaussiana
- Piccolo $\sigma \rightarrow$ funzione molto stretta, ogni punto influenza solo la sua zona, rischio di **overfitting**
- Grande $\sigma \rightarrow$ funzione molto larga, i punti influenzano regioni più ampie, rischio di **underfitting**

4.6 Scelta dei landmarks

I landmarks sono i punti del dataset rispetto ai quali si calcola la similarità gaussiana:

- Spesso si scelgono tutti i punti di training come landmarks
- In alternativa si possono scegliere solo una selezione (es. clustering k-means) per ridurre il numero di landmarks [utile per ridurre il costo computazionale]

5 Regularization

5.1 Holdout Method

Separo il set originale in Training Set (Training + Validation) ed il Test Set. Il Validation serve a migliorare man mano il modello che si basa sul training set, mentre il test set viene usato solo per la stima della performance finale.

5.1.1 K-fold Cross Validation

Viene usato per migliorare l'holdout method. Invece di fare un solo esperimento ne faccio molteplici. Prendo il training set e lo divido in k parti uguali; man mano useremo $k-1$ parti per il training e 1 per validation cambiando ogni volta il gruppo singolo in modo da usare alla fine tutti e k i pezzi del training set come validation. Una volta fatto ciò possiamo calcolare tutte e k le performance diversi e posso fare una media delle performance.

5.2 Overfitting e Underfitting

- **Overfitting:** Con troppe features l'ipotesi imparata potrebbe essere molto buona con i dati del training set ma fallisce nel generalizzare nuovi esempi
- **Underfitting:** Il modello è troppo semplice per catturare bene il pattern dei dati. Non impara bene e avrà scarse performance sia su training che test set.

5.3 Ridge Regression

Problema di minimizzazione:

$$\min J(\theta) = \frac{1}{2m} \cdot \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \cdot \sum_{j=1}^n \theta_j^2 \right]$$

5.4 Regularizzazione della Logistic Regression

Problema di minimizzazione:

$$\min J(\theta) = \frac{1}{m} \left[\sum_{i=1}^m \text{Cost}(h_{\theta}(x), y) + \frac{\lambda}{2} \cdot \sum_{j=1}^n \theta_j^2 \right]$$

5.5 Learning Curve

- La learning Curve nel caso di **alto bias** presenta un gap piccolo tra training e validation error
- La learning Curve nel caso di **alta varianza** presenta un largo gap tra training e validation error

5.6 Fix da provare

- Prova ad aggiungere training examples, ad usare un set più piccolo di features oppure ad aumentare λ **per fixare una varianza troppo alta**
- Prova ad aumentare il set di features o a diminuire λ **per fixare un bias troppo alto**

6 Dimensionality Reduction

Tecnica di riduzione della dimensionalità che trasforma i dati in un nuovo insieme di variabili (componenti principali), ottenute come combinazioni lineari delle originali, mantenendo la varianza spiegata e riducendo le ridondanze. Per portare da n a k dimensioni bisogna trovare k vettori.

6.1 Differenza Linear Regression e PCA

La differenza sta nel come viene calcolata la distanza tra il punto ed il modello.

- **PCA:** Si misura l'errore come differenza tra il valore effettivo e la predizione nello stesso punto (stessa feature)
- **Linear Regression:** si misura perpendicolarmente al modello (non considero le features)

6.2 Funzionamento Algoritmo

- **Data Preprocessing** con feature scaling o mean normalization sostituendo ogni $x_j^{(i)}$ con $x_j - \mu_j$
- Calcolo la **Matrice di Covarianza** $\Sigma = \frac{1}{m} \cdot \sum_{i=1}^n (x^{(i)}) \cdot (x^{(i)})^T$
- Calcolo degli **autovettori** (per trovare i vettori più importanti) della matrice Σ : $[U, S, V] = \text{svd}(\Sigma)$ dove per svd si intende la singular value decomposition
- Definisco $U_{\text{reduce}} = U(:, 1:k)$ quindi prendo i k autovettori più importanti.
- Proietto i punti sul nuovo spazio $z = U_{\text{reduce}} \cdot x$

6.3 Scelta di k

Si usa la formula dell' **average squared projection error**. Si sceglie il più piccolo k tale che:

$$\frac{\frac{1}{m} \cdot \sum_{i=1}^m \|x^{(i)} - x_{approx}\|^2}{\frac{1}{m} \cdot \sum_{i=1}^m \|x^{(i)}\|^2} \leq 0.01 \rightarrow 1\% \quad 99\% \text{ della varianza mantenuta}$$